

Informe Técnico de Incidente

Signo de latitud GPS perdido en el envío al servidor

Sensor de monitoreo de espectro SDR — nodo **ane7-pi** (Leticia, Amazonas)

Tipo de artefacto:	Informe de diagnóstico y corrección
Severidad:	Alta (ubicación reportada ~900 km al norte)
Componentes afectados:	<code>gps-lte/libs/utils.c</code> , <code>gps-lte/gps-lte.c</code>
Componentes descartados:	Backend, base de datos, frontend (mapa)
Fecha del documento:	27 de julio de 2026

Este informe documenta el diagnóstico, la causa raíz, la corrección y la verificación del defecto por el cual la plataforma web ubicaba el sensor en el hemisferio norte (Vichada/Cumaribo, 4.1964 N) cuando el dispositivo se encuentra en Leticia, Amazonas (−4,1963, −69,9410). El análisis se basa en inspección directa del código fuente del repositorio y en evidencia capturada en el dispositivo en producción.

Capítulo 1

Resumen ejecutivo

La plataforma mostraba el sensor ~ 900 km al norte de su posición real porque el **propio sensor transmitía la latitud sin signo**. El binario `ltegps_app` enviaba `"lat": 4.196348` (positivo) en el POST HTTP al endpoint `/gps`, mientras que internamente sí calculaba el valor correcto (-4.1963464) para su memoria compartida. La causa raíz está en la función `post_gps_data()` (`gps-lte/libs/utlis.c`), que convertía las coordenadas NMEA a grados decimales sin recibir ni aplicar el indicador de hemisferio (N/S/E/W). Backend, base de datos y frontend quedan **descartados** como causa: el servidor respondía `{"success":true}` almacenando fielmente el valor (incorrecto) recibido.

La corrección ya está implementada y verificada: se extiende `post_gps_data()` para recibir los indicadores de dirección NMEA y aplicar el signo correspondiente (S/W $\rightarrow$ negativo). Tres archivos modificados, compilación y enlace verificados, y prueba unitaria de la conversión superada con los valores reales del log del dispositivo.

Capítulo 2

Diagnóstico

2.1. Síntoma reportado

- Posición real del dispositivo: Leticia, Amazonas ($-4,1963$, $-69,9410$).
- Posición mostrada en la plataforma: Vichada/Cumaribo ($+4,1964$, $-69,9410$).
- La longitud era correcta; solo la latitud perdía el signo negativo.

2.2. Las dos rutas de la coordenada dentro del sensor

La trama NMEA cruda del módulo GPS transporta campos *sin signo*; el hemisferio se indica únicamente con una letra (N/S para latitud, E/W para longitud):

```
Jul 27 12:38:39 ane7-pi ltegps-ane2[827]: Latitude: 0411.780345, Longitude:
06956.463437, Altitude: 124.5
```

Es decir, $0411.780345 = 4^{\circ} 11,780345' = 4,1963^{\circ}$, y la letra **S** (parseada en `GPSInfo.LatDir`) es la única portadora del signo. A partir de ese dato crudo, el código tenía dos rutas divergentes:

Ruta A: memoria compartida (correcta)	Ruta B: POST HTTP (defectuosa)
<code>gps_to_decimal()</code> (<code>gps-lte.c:73</code>) recibe <code>GPSInfo.LatDir</code> y aplica <code>S → -fabs(val)</code> .	<code>post_gps_data()</code> (<code>utils.c:160</code>) recibía solo las cadenas NMEA crudas, sin el indicador de dirección.
Escribe <code>/dev/shm/persistent.json</code> : <code>"last_lat":-4.1963464</code> (formato <code>%.7f</code>).	Transmitía <code>"lat": 4.196346</code> (formato <code>%.6f</code>) al endpoint <code>/gps</code> .

2.3. Causa raíz

En `gps-lte/gps-lte.c:415` el sitio de llamada no pasaba los campos de dirección (`GPSInfo.LatDir`, `GPSInfo.LonDir`), que sí existen y ya estaban parseados. Dentro de `post_gps_data()`:

```
double final_lat = nmea_to_decimal(raw_lat); // ddmm.mmmm -> dd.ddd, SIN signo
double final_lng = nmea_to_decimal(raw_lng);

// Correccion de Hemisferio (Oeste para Colombia)
if (final_lng > 0) {
    final_lng = -final_lng; // hack fijo: solo longitud
```

```
}  
// Latitud: NUNCA se corregia
```

La longitud se salvaba por un hack codificado a mano (“Colombia está al oeste”), pero la latitud quedaba positiva para cualquier ubicación del hemisferio sur. La aritmética confirma el síntoma con exactitud: $\text{atof}("0411.7808") = 411,7808$; $\text{nmea_to_decimal} = 4 + 11,7808/60 = +4,1964$, el valor erróneo observado en la plataforma.

2.4. Por qué la evidencia de `strace` parecía exonerar al sensor

Se capturó en el dispositivo:

```
sudo strace -e trace=read -e read=N -s 256 -p 827  
read(4, "{\"last_lat\":-4.1963464,\"last_lng\":-69.9410436,\"delta_t_ms\":849,\"  
      campaign_id\":315,...,\"window\":\"hamming\",\"lna_gain\":40,...\", 4095) = 450  
read(4, "", 3645) = 0
```

Esto **no** es tráfico de red. Es la lectura del archivo local `/dev/shm/persistent.json` (tmpfs), por tres razones independientes:

1. **Los flags:** se trazaron solo llamadas `read()` sobre un descriptor de archivo; nunca `sendto()/write()` del socket.
2. **El contenido:** el JSON incluye claves de campaña y RF (`center_freq_hz`, `lna_gain`, `window`, `overlap`) que escribe el orquestador Python vía `ShmStore`. Ningún payload HTTP del sensor contiene parámetros de RF; el POST a `/gps` solo lleva `mac`, `lat`, `lng`, `alt`.
3. **El patrón de lectura:** `read(..., 4095) = 450` seguido de `read(..., 0)` (EOF) corresponde a `read_all_from_fd_gps()` (`utils.c:16`), invocado por `shm_consult_persistent_gps()` en el bloque de histéresis (`gps-lte.c:433--434`) en cada ciclo.

Además, la URL de la API es HTTPS (`https://rsm.ane.gov.co:12443`): aun trazando el socket, TLS cifra el cuerpo antes del `syscall`; el payload real jamás es visible en `strace`. El único JSON en claro visible en los `syscalls` de este proceso es el del archivo de memoria compartida.

2.5. Evidencia definitiva en el dispositivo

`post_gps_data()` imprime exactamente lo que transmite (`utils.c:247`). El journal del servicio mostró:

```
Jul 27 12:47:05 ane7-pi ltegps-ane2[827]: {"success":true,"message":"GPS data received  
"}[UTILS] Datos GPS (4.196348, -69.941085, 124.3)enviados con exito.
```

Y simultáneamente, el archivo de memoria compartida:

```
$ cat /dev/shm/persistent.json  
{\"last_lat\":-4.1963464,\"last_lng\":-69.9410436,...,\"changed_gps\":false}
```

Mismo dispositivo, mismo instante: **transmitido** `+4,196348` (erróneo), **almacenado localmente** `-4,1963464` (correcto). Las dos rutas, capturadas en vivo. La respuesta `success:true` del servidor confirma que el backend almacena fielmente lo recibido: la pérdida del signo ocurre en el sensor.

Capítulo 3

Corrección implementada

Cambio quirúrgico en tres archivos (rama de trabajo, sin efectos colaterales):

- `gps-lte/libs/utils.h` — declaración de `post_gps_data()` extendida con `lat_dir_str` y `lon_dir_str`, documentación actualizada.
- `gps-lte/libs/utils.c` — el signo se deriva del indicador NMEA (S/W → `-fabs()`, N/E → `fabs()`), con `toupper()` para robustez. Se añadió `fabs()` defensivo sobre el valor crudo y se conserva el fallback “Colombia al oeste” *únicamente* cuando el indicador de longitud está ausente, para no regresionar el caso sin dirección.
- `gps-lte/gps-lte.c:415` — el sitio de llamada ahora pasa `GPSInfo.LatDir` y `GPSInfo.LonDir`.

```
double final_lat = nmea_to_decimal(fabs(raw_lat));
double final_lng = nmea_to_decimal(fabs(raw_lng));

// Correccion de hemisferio segun indicador N/S/E/W de la trama NMEA
if (lat_dir_str != NULL && lat_dir_str[0] != '\0') {
    char dir = (char)toupper((unsigned char)lat_dir_str[0]);
    final_lat = (dir == 'S') ? -fabs(final_lat) : fabs(final_lat);
}
if (lon_dir_str != NULL && lon_dir_str[0] != '\0') {
    char dir = (char)toupper((unsigned char)lon_dir_str[0]);
    final_lng = (dir == 'W') ? -fabs(final_lng) : fabs(final_lng);
} else if (final_lng > 0) {
    final_lng = -final_lng; // fallback sin indicador: hemisferio occidental
}
```

Capítulo 4

Verificación

4.1. Compilación

Nota de entorno: `./build.sh -dev` compila *únicamente* `rf_app` (el modo `BUILD_STANDALONE` de CMake no define el objetivo `ltegps_app`), por lo que no sirve para verificar cambios de GPS. El binario `ltegps_app` requiere `gpiod` (solo disponible en la Raspberry Pi). Se verificó compilación y enlace completos en escritorio compilando manualmente con cabeceras *stub* para `gpiod/curl/cJSON`:

Resultado	Detalle
<code>utils.c</code> , <code>gps-lte.c</code>	Compilación limpia con <code>-Wall -Wextra</code> , 0 advertencias.
Enlace <code>ltegps_app</code> completo	Exitoso (<code>-lcurl -lcjson -lm -lpthread</code>).
Advertencias restantes	Preexistentes en <code>bacn_GPS.c</code> , <code>bacn_LTE.c</code> , <code>bacn_gpio.c</code> (no modificados).

4.2. Prueba unitaria de la conversión

Se ejercitó la lógica corregida con los valores reales del log del dispositivo y los cuatro cuadrantes de hemisferio:

Caso	Entrada NMEA	Resultado	OK
Leticia (S/W, dato real)	0411.781722,S 06956.463968,W	-4,1963620, -69,9410661	✓
Vichada (N/W)	0606.000000,N 06955.000000,W	+6,1000000, -69,9166667	✓
Bogotá (N/W)	0436.000000,N 07405.000000,W	+4,6000000, -74,0833333	✓
Sin dirección (fallback)	0411.781722 06956.463968	lng -69,9410661	✓
Minúsculas (s/w)	0411.781722,s	-4,1963620	✓

4.3. Despliegue y verificación final (en la Raspberry Pi)

```
sudo ./build.sh # modo estandar (incluye gpiod y ltegps_app)
sudo systemctl restart ltegps-ane2.service
journalctl -u ltegps-ane2.service -f | grep "Datos_GPS"
```

Criterio de aceptación: la línea debe mostrar latitud negativa en el primer ciclo de envío posterior al reinicio.

Binario	Salida esperada en journal
Actual (defectuoso)	Datos GPS (4.196348, -69.941085, 124.3)
Corregido	Datos GPS (-4.196348, -69.941085, 124.3)

La plataforma corregirá la posición del marcador con el primer POST aceptado tras el reinicio; no se requieren cambios en backend, base de datos ni frontend. Los registros históricos erróneos permanecen, pero la posición en vivo se autocorrigie.

Capítulo 5

Recomendaciones

1. **Eliminar la duplicación de conversores NMEA.** Coexisten `nmea_to_decimal()` (`utils.c`) y `gps_to_decimal()` (`gps-lte.c`) con semántica divergente; esa divergencia es el origen de esta clase de error. Consolidar en una única función compartida.
2. **Eliminar hacks geográficos codificados a mano.** La regla fija “longitud positiva → negar” asume Colombia al oeste; la fuente de verdad debe ser siempre el indicador N/S/E/W de la trama.
3. **Validación en servidor (defensa en profundidad).** Aunque no fue la causa, el backend podría registrar advertencia cuando un sensor reporte una coordenada fuera del ámbito nacional esperado, como detección temprana de anomalías de ingesta.
4. **Método de verificación de payload.** Para auditar lo que el sensor transmite, usar el `printf` de `post_gps_data()` vía `journalctl -u ltegps-ane2.service | grep "Datos GPS"`; `strace` no es apto porque el tráfico es TLS y el único JSON en claro visible es el de `/dev/shm/persistent.json`.